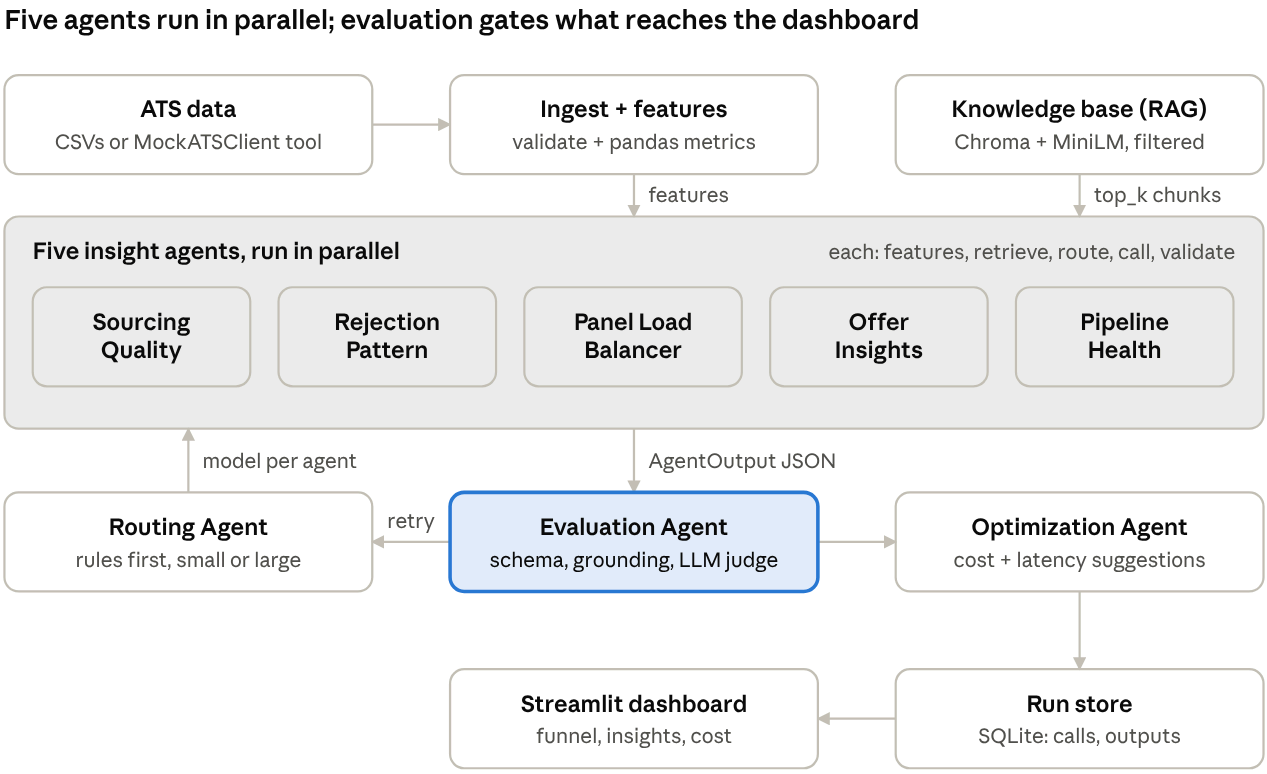
Hiring Intelligence System — Python Implementation Guide

Oct 18, 2026 · @Aravinda

Overview and architecture decisions

Build it as a code-first Python system: LangGraph orchestrates 8 agents, LiteLLM gives one API across small and large models, Chroma holds the RAG layer, and Streamlit shows the results. Every agent returns the same Pydantic contract, so evaluation, cost tracking and the dashboard all consume one shape.



system architecture · 8 agents, RAG, run store

Any agent scoring under 0.6 goes back through the Routing Agent once, forced to the large model; everything else flows on to optimization, the run store and the dashboard.

Why LangGraph over n8n. You need parallel fan-out to five agents, shared typed state, per-node error capture and a conditional retry loop after evaluation. LangGraph gives all four in plain Python, which you can unit-test and diff. n8n is faster to click together but

harder to version, test and explain in the reflection doc.

Why LiteLLM. The Routing Agent must swap models per task. LiteLLM exposes OpenAI, Anthropic, Gemini and local Ollama models behind one `completion()` call and returns token usage, so routing is a string change, not a new client.

Stack

Concern	Choice	Why
Orchestration	LangGraph	Parallel nodes, typed state, conditional edges
LLM access	LiteLLM	One API, many providers, usage per call
Structured output	Pydantic v2	Schema validation before anything reaches the UI
Embeddings	sentence-transformers `all-MiniLM-L6-v2`	Local, free, 384-dim, fast enough at this scale
Vector DB	Chroma (persistent, local)	Metadata filters per agent, zero infra
Data	pandas + SQLite	CSV ingest, run log, cost log
Dashboard	Streamlit + Plotly	Funnel, insights, cost and latency in one page
Logging	Python `logging` with JSON formatter	One line per agent call, greppable by `run_id`
Tests	pytest	Contract tests, deterministic metric tests

Repo layout

```
hiring-intel/
├── pyproject.toml / requirements.txt
├── .env.example                # OPENAI_API_KEY, ANTHROPIC_API_KEY
├── config/
│   ├── models.yaml             # model tiers + USD prices per 1M tokens
│   └── sla.yaml                # stage SLA targets in days
├── data/
```

```
|   ├── raw/                      # generated mock ATS CSVs
|   ├── knowledge/                # markdown docs for RAG
|   └── golden/golden_set.jsonl   # 15-20 evaluation scenarios
├── hiring_intel/
|   ├── contracts.py              # shared output contract
|   ├── llm.py                    # LiteLLM wrapper: cost, latency, retries,
cache
|   ├── ingest.py                 # load, clean, validate CSVs
|   ├── metrics.py                # deterministic pandas features per agent
|   ├── rag/                      # build_index.py, retriever.py
|   ├── agents/
|   |   ├── base.py               # BaseInsightAgent
|   |   ├── sourcing.py  rejection.py  panel_load.py
|   |   ├── offers.py    pipeline_health.py
|   |   ├── router.py    evaluator.py  optimizer.py
|   ├── graph.py                  # LangGraph orchestrator
|   ├── observability.py          # JSON logs + SQLite run store
|   └── prompts/                  # one .md prompt per agent, versioned
├── scripts/
|   ├── generate_mock_data.py
|   ├── run_pipeline.py
|   ├── run_eval.py
|   └── benchmark_levers.py       # before/after optimization report
├── app/dashboard.py              # Streamlit
├── tests/
└── docs/                         # architecture, contracts, eval,
optimization, reflection
```

One principle runs through every step: **compute numbers in pandas, let the LLM interpret them.** The agents never count rows themselves. That single rule removes most hallucination risk and makes the evidence field verifiable.

Step 1 — Environment setup

Use Python 3.11+ and a virtual environment; the full install is about 10 packages.

```
mkdir hiring-intel && cd hiring-intel
python -m venv .venv && source .venv/bin/activate
pip install langgraph litellm pydantic pandas numpy faker chromadb \
    sentence-transformers streamlit plotly pyyaml python-dotenv pytest
tabulate
```

`config/models.yaml` holds model tiers and prices. Prices below are examples; copy current ones from your provider's pricing page before you report USD.

```
tiers:
  small:
    model: gpt-4o-mini          # or anthropic/claude-haiku-*,
ollama/llama3.1
    input_per_1m: 0.15
    output_per_1m: 0.60
  large:
    model: gpt-4o               # or anthropic/claude-sonnet-*
    input_per_1m: 2.50
    output_per_1m: 10.00
judge:
  model: gpt-4o                 # judge should differ from, or be >=, the
generator
```

`config/sla.yaml` holds the targets the Pipeline Health Agent checks against:

```
stage_sla_days: {applied: 3, screen: 5, tech_1: 7, tech_2: 7, onsite: 7,
offer: 5}
max_days_open: {default: 45, senior: 60}
max_interviews_per_week: 4
```

Step 2 — Mock ATS data with planted patterns

Generate synthetic data with **deliberately planted patterns**, so you know the right answers before any agent runs. Those planted answers become your golden set in Step 8.

Six tables, joined by IDs:

Table	Key columns
`requisitions.csv`	req_id, role, level, team, opened_at, closed_at, status
`candidates.csv`	candidate_id, req_id, source, applied_at, current_stage, outcome
`stage_events.csv`	candidate_id, stage, entered_at, exited_at, result, rejection_reason

Table	Key columns
`interviews.csv`	interview_id, candidate_id, interviewer_id, stage, scheduled_at, score
`interviewers.csv`	interviewer_id, name, team, level
`offers.csv`	candidate_id, req_id, extended_at, responded_at, status, decline_reason, offered_ctc, expected_ctc

Patterns to plant (write them in `docs/planted_patterns.md`):

1. Referrals convert to hire at about 3x job boards for Backend roles.
2. Agency candidates for Data roles fail `tech_1` at about 70%, reason "fundamentals".
3. Mobile roles lose about 80% at `screen` with reason "JD mismatch" (a JD problem, not an interview problem).
4. Two interviewers carry over 35% of all interviews; five carry under 3%.
5. Offer declines cluster on "compensation" when offered_ctc is under 90% of expected_ctc.
6. `onsite` scheduling breaches SLA for SRE roles; two reqs have been open over 90 days.

`scripts/generate_mock_data.py` (core of it):

```
import random, numpy as np, pandas as pd
from datetime import datetime, timedelta
from faker import Faker

rng = np.random.default_rng(42); random.seed(42); fake = Faker();
Faker.seed(42)
ROLES = ["Backend", "Mobile", "Data", "SRE", "Frontend"]
SOURCES = ["referral", "linkedin", "job_board", "agency", "careers_page"]
STAGES = ["applied", "screen", "tech_1", "tech_2", "onsite", "offer"]
START = datetime(2026, 4, 1)

def pass_prob(role, source, stage):
    p = 0.6
    if source == "referral" and role == "Backend": p += 0.25        # pattern
1
    if source == "job_board": p -= 0.15
    if source == "agency" and role == "Data" and stage == "tech_1": p = 0.3
# pattern 2
    if role == "Mobile" and stage == "screen": p = 0.2           # pattern 3
```

```
    return max(0.05, min(p, 0.95))

def rejection_reason(role, source, stage):
    if role == "Mobile" and stage == "screen": return "JD mismatch"
    if role == "Data" and source == "agency" and stage == "tech_1": return
"fundamentals"
    return random.choice(["communication", "system design", "culture add",
"coding", "experience gap"])

# interviewers: skewed load for pattern 4
interviewers = pd.DataFrame([{"interviewer_id": f"I{i:02d}", "name":
fake.name(),
                              "team": random.choice(ROLES), "level":
random.choice(["SE2", "SSE", "Staff"])}
                             for i in range(20)])
load_weights = np.array([12, 11] + [1.0] * 13 + [0.1] * 5); load_weights /=
load_weights.sum()

reqs, cands, events, interviews, offers = [], [], [], [], []
for r in range(30):
    role = ROLES[r % len(ROLES)]
    opened = START + timedelta(days=int(rng.integers(0, 120)))
    if role == "SRE" and r < 12: opened = START - timedelta(days=40)      #
pattern 6: long-open reqs
    reqs.append({"req_id": f"R{r:03d}", "role": role, "level":
random.choice(["mid", "senior"]),
                 "team": role, "opened_at": opened, "closed_at": None,
"status": "open"})
    for c in range(int(rng.integers(25, 60))):
        cid, source = f"C{r:03d}{c:03d}", random.choice(SOURCES)
        t = opened + timedelta(days=int(rng.integers(0, 30)))
        outcome, stage_reached = "in_progress", "applied"
        for stage in STAGES:
            sla_mult = 2.5 if (role == "SRE" and stage == "onsite") else 1.0
# pattern 6
            dwell = max(1, int(rng.gamma(2, 2) * sla_mult))
            passed = rng.random() < pass_prob(role, source, stage)
            events.append({"candidate_id": cid, "stage": stage, "entered_at":
t,
                           "exited_at": t + timedelta(days=dwell), "result":
"pass" if passed else "reject",
                           "rejection_reason": None if passed else
rejection_reason(role, source, stage)})
```

```
            if stage not in ("applied", "offer"):
                interviews.append({"interview_id": f"{cid}-{stage}",
"candidate_id": cid,
                                   "interviewer_id":
rng.choice(interviewers.interviewer_id, p=load_weights),
                                   "stage": stage, "scheduled_at": t +
timedelta(days=dwell - 1),
                                   "score": int(rng.integers(1, 5))})
            t += timedelta(days=dwell); stage_reached = stage
            if not passed: outcome = "rejected"; break
        if outcome != "rejected" and stage_reached == "offer":
            expected = int(rng.integers(25, 60)) * 1e5
            offered = expected * rng.uniform(0.8, 1.1)
            low = offered < 0.9 * expected
            accepted = rng.random() < (0.25 if low else 0.8)
# pattern 5
            offers.append({"candidate_id": cid, "req_id": f"R{r:03d}",
"extended_at": t,
                           "responded_at": t +
timedelta(days=int(rng.integers(1, 10))),
                           "status": "accepted" if accepted else "declined",
                           "decline_reason": None if accepted else
("compensation" if low else
                                                 random.choice(["competing
offer", "timing", "location"])),
                           "offered_ctc": round(offered), "expected_ctc":
round(expected)})
            outcome = "hired" if accepted else "offer_declined"
        cands.append({"candidate_id": cid, "req_id": f"R{r:03d}", "source":
source,
                      "applied_at": opened, "current_stage": stage_reached,
"outcome": outcome})

for name, rows in [("requisitions", reqs), ("candidates", cands),
("stage_events", events),
                   ("interviews", interviews), ("offers", offers)]:
    pd.DataFrame(rows).to_csv(f"data/raw/{name}.csv", index=False)
interviewers.to_csv("data/raw/interviewers.csv", index=False)
```

That yields roughly 1,200 candidates and a few thousand stage events: enough for real patterns, small enough to iterate fast.

Ingest (`ingest.py`). Load each CSV, parse dates, drop exact duplicates, check referential integrity, and return a `HiringData` dataclass. Write rejected rows to `data/quarantine/` instead of failing the run. Expose the same loader behind a `MockATSClient.get_candidates(req_id=None)` class so agents can call it as a **tool**, and a real ATS API can replace it later.

Step 3 — Shared output contract and the LLM client

Define the contract first; every other module depends on it. The LLM fills only the reasoning fields, and your code fills `cost_of_insight` from real token counts, so cost can never be hallucinated.

`hiring_intel/contracts.py`

```
from typing import Literal
from pydantic import BaseModel, Field, field_validator

class Evidence(BaseModel):
    metric: str                       # e.g. "tech_1_pass_rate"
    value: float | str
    segment: dict[str, str] = {}      # {"role": "Data", "source": "agency"}
    source: Literal["data", "rag"]    # computed from CSV, or retrieved
benchmark
    ref: str | None = None            # RAG chunk id, for traceability

class Cost(BaseModel):
    model: str
    input_tokens: int
    output_tokens: int
    usd: float
    latency_ms: int

class Alternative(BaseModel):
    option: str                       # "Run on small model" / "Rule-based
only"
    trade_off: str
    est_usd: float | None = None

class Insight(BaseModel):
    """One actionable finding. Agents return 1-3 of these."""
    title: str
```

```
    recommendation: str = Field(min_length=30)
    evidence: list[Evidence] = Field(min_length=1)
    confidence_score: float = Field(ge=0, le=1)
    alternative: Alternative
    severity: Literal["low", "medium", "high"] = "medium"

    @field_validator("recommendation")
    @classmethod
    def must_be_actionable(cls, v: str) -> str:
        vague = ("consider improving", "monitor closely", "it may be")
        if any(p in v.lower() for p in vague):
            raise ValueError("recommendation is generic; name an owner,
action or number")
        return v

class AgentOutput(BaseModel):
    agent: str
    run_id: str
    status: Literal["ok", "degraded", "failed"] = "ok"
    insights: list[Insight] = []
    cost_of_insight: Cost | None = None   # filled by code, never by the LLM
    retrieved_chunks: list[str] = []      # ids used for grounding
    error: str | None = None

class LLMInsights(BaseModel):
    """What the LLM is asked to return. Subset of AgentOutput."""
    insights: list[Insight]
```

The five required fields map one to one: `recommendation`, `evidence`, `confidence_score`, `cost_of_insight` and `alternative`. Agent-specific fields (for example `rebalance_plan` on the Panel Load Balancer) extend `Insight` in a subclass.

`hiring_intel/llm.py` — one door for every model call

This wrapper does five jobs: calls the model, validates JSON against a Pydantic schema, repairs once on failure, measures tokens/latency/USD, and caches identical calls.

```
import hashlib, json, time, yaml, litellm
from pydantic import BaseModel, ValidationError
from hiring_intel.contracts import Cost
from hiring_intel.observability import log_call

CFG = yaml.safe_load(open("config/models.yaml"))
```

```
PRICES = {t["model"]: t for t in CFG["tiers"].values()}
_cache: dict[str, tuple] = {}

def _usd(model: str, inp: int, out: int) -> float:
    p = PRICES.get(model)
    if not p:                                    # fall back to LiteLLM's
price table
        return sum(litellm.cost_per_token(model=model, prompt_tokens=inp,
completion_tokens=out))
    return inp / 1e6 * p["input_per_1m"] + out / 1e6 * p["output_per_1m"]

def call_structured(*, agent: str, run_id: str, model: str, system: str,
user: str,
                    schema: type[BaseModel], temperature: float = 0.1,
                    use_cache: bool = True, max_repairs: int = 1) ->
tuple[BaseModel, Cost]:
    key = hashlib.sha256(f"{model}|{system}|{user}".encode()).hexdigest()
    if use_cache and key in _cache:
        parsed, cost = _cache[key]
        log_call(agent, run_id, cost, cached=True)
        return parsed, cost.model_copy(update={"usd": 0.0, "latency_ms": 0})

    messages = [{"role": "system", "content": system + "\n\nReturn ONLY JSON
matching this schema:\n"
                 + json.dumps(schema.model_json_schema())},
                {"role": "user", "content": user}]
    inp = out = 0; t0 = time.perf_counter()
    for attempt in range(max_repairs + 1):
        resp = litellm.completion(model=model, messages=messages,
temperature=temperature,
                                  response_format={"type": "json_object"},
num_retries=2, timeout=60)
        inp += resp.usage.prompt_tokens; out += resp.usage.completion_tokens
        raw = resp.choices[0].message.content
        try:
            parsed = schema.model_validate_json(raw)
            break
        except ValidationError as e:
            if attempt == max_repairs:
                raise
            messages += [{"role": "assistant", "content": raw},
                         {"role": "user", "content": f"Your JSON failed
validation: {e}. Fix it and return JSON only."}]
```

```
    cost = Cost(model=model, input_tokens=inp, output_tokens=out,
usd=round(_usd(model, inp, out), 6),
                latency_ms=int((time.perf_counter() - t0) * 1000))
    _cache[key] = (parsed, cost)
    log_call(agent, run_id, cost, cached=False)
    return parsed, cost
```

The YAML prices win over LiteLLM's built-in table, so the USD figures in your optimization report trace to a price list you can cite.

`hiring_intel/observability.py`

Log one JSON line per call and persist every call to SQLite; the dashboard and the Optimization Agent both read this table.

```
import json, logging, sqlite3, datetime as dt

logging.basicConfig(level=logging.INFO, format="%(message)s")
log = logging.getLogger("hiring_intel")
DB = sqlite3.connect("data/runs.db", check_same_thread=False)
DB.execute("""CREATE TABLE IF NOT EXISTS llm_calls(ts TEXT, run_id TEXT,
agent TEXT, model TEXT,
              input_tokens INT, output_tokens INT, usd REAL, latency_ms INT,
cached INT, variant TEXT)""")
DB.execute("""CREATE TABLE IF NOT EXISTS agent_outputs(run_id TEXT, agent
TEXT, status TEXT, json TEXT)""")
DB.execute("""CREATE TABLE IF NOT EXISTS routing(run_id TEXT, agent TEXT,
tier TEXT, model TEXT, reason TEXT)""")

def log_call(agent, run_id, cost, cached, variant="baseline"):
    row = dict(ts=dt.datetime.utcnow().isoformat(), run_id=run_id,
agent=agent, cached=int(cached),
               variant=variant, **cost.model_dump())
    log.info(json.dumps({"event": "llm_call", **row}))
    DB.execute("INSERT INTO llm_calls VALUES
(:ts,:run_id,:agent,:model,:input_tokens,:output_tokens,"
               ":usd,:latency_ms,:cached,:variant)", row); DB.commit()
```

If you want a hosted trace view, add Langfuse through LiteLLM's callback (`litellm.success_callback = ["langfuse"]`). It is optional; SQLite already covers the brief.

Step 4 — RAG knowledge base (Chroma + MiniLM)

Ground each agent in a small, tagged knowledge base: local MiniLM embeddings, Chroma on disk, top_k = 4, a cosine-distance cutoff of 0.6, and metadata filters per agent. At this scale (a few hundred chunks) that is free, runs in milliseconds, and is easy to explain.

4.1 Write the knowledge base

Create 15–25 short markdown files under `data/knowledge/`, each with YAML front matter for filters. Mark every benchmark as illustrative in your docs, since it is authored by you.

```
---
doc_type: benchmark          # benchmark | sla | past_insight |
rejection_history | offer_history | playbook
agents: [sourcing]
role: Backend
period: 2026-Q1
---
# Sourcing benchmark — Backend
Referral-to-hire conversion: 8-12%. Job-board conversion: 2-4%.
Agency candidates cost ~15% of first-year CTC; justify only for niche skills.
```

Suggested set: sourcing benchmarks per role (5), stage SLA policy (1), last two quarters' rejection summaries (4), offer outcome history and comp bands (3), interviewer load policy (1), structured-interview playbook (2), past insight memos with what was done and the result (4).

4.2 Build the index — `hiring_intel/rag/build_index.py`

```
import pathlib, yaml, chromadb
from chromadb.utils import embedding_functions

EMB =
embedding_functions.SentenceTransformerEmbeddingFunction(model_name="all-
MiniLM-L6-v2")
client = chromadb.PersistentClient(path="data/chroma")

def chunk(text: str, size: int = 600, overlap: int = 80) -> list[str]:
    out, i = [], 0
    while i < len(text):
        out.append(text[i:i + size]); i += size - overlap
```

```
    return out

def build():
    client.delete_collection("hiring_kb") if "hiring_kb" in [c.name for c in
client.list_collections()] else None
    col = client.create_collection("hiring_kb", embedding_function=EMB,
metadata={"hnsw:space": "cosine"})
    ids, docs, metas = [], [], []
    for f in pathlib.Path("data/knowledge").glob("*.md"):
        _, fm, body = f.read_text().split("---", 2)
        meta = yaml.safe_load(fm)
        for j, c in enumerate(chunk(body.strip())):
            ids.append(f"{f.stem}#{j}"); docs.append(c)
            metas.append({"doc_type": meta["doc_type"], "role":
meta.get("role", "all"),
                          "agents": ",".join(meta["agents"]), "period":
meta.get("period", ""),
                          **{f"agent_{a}": True for a in meta["agents"]}})
    col.add(ids=ids, documents=docs, metadatas=metas)
    print(f"indexed {len(ids)} chunks")

if __name__ == "__main__":
    build()
```

Chroma metadata values must be scalars, so each agent tag becomes its own boolean key (`agent_sourcing: True`). That makes the per-agent filter a simple equality.

Also index the **output of past runs**: after each run, add accepted insights as `doc_type: past_insight`. The next run can then say "same pattern as last month; the fix was X".

4.3 Retriever with filters, threshold and cache — `hiring_intel/rag/retriever.py`

```
from functools import lru_cache
import chromadb
from chromadb.utils import embedding_functions

EMB =
embedding_functions.SentenceTransformerEmbeddingFunction(model_name="all-
MiniLM-L6-v2")
COL =
chromadb.PersistentClient(path="data/chroma").get_collection("hiring_kb",
embedding_function=EMB)
MAX_DISTANCE = 0.6        # cosine distance; tune on 10 hand-labelled queries

@lru_cache(maxsize=256)   # optimization lever: repeated queries skip
embedding + search
def retrieve(query: str, agent: str, role: str | None = None, k: int = 4) ->
tuple[tuple[str, str], ...]:
    where = {"agent_" + agent: True}
    if role:
        where = {"$and": [where, {"role": {"$in": [role, "all"]}}]}
    res = COL.query(query_texts=[query], n_results=k, where=where)
    hits = [(i, d) for i, d, dist in zip(res["ids"][0], res["documents"][0],
res["distances"][0])
            if dist <= MAX_DISTANCE]
    return tuple(hits)    # tuple so lru_cache can store it
```

Agent	Filter	Typical query
Sourcing Quality	`agent_sourcing`, role	"source conversion benchmark {role}"
Rejection Pattern	`agent_rejection`, role	"rejection history {stage} {role}"
Panel Load	`agent_panel_load`	"interviewer load policy"
Offer Insights	`agent_offers`, role	"offer decline compensation band {role}"
Pipeline Health	`agent_pipeline`	"stage SLA targets time to fill"

Tuning top_k and the threshold. Write 10 queries with the chunk you expect back, then measure hit@k for k = 2, 4, 6 and thresholds 0.5–0.7. Put that small table in `docs/eval_notes.md`; it answers the brief's "what top_k and threshold?" question with

data.

Grounding rule in prompts. Every retrieved chunk is passed as `[chunk_id] text`, and any evidence item with `source: "rag"` must cite a `ref` from that list. The Evaluation Agent checks this, which is how you prove at least two agents are grounded.

Step 5 — Routing Agent

Route with a scored rule set first and ask a small LLM only when the score is borderline. A rule costs nothing and is explainable; paying a model to pick a model for every call would eat the savings you are trying to show.

Complexity features

Each insight agent publishes a `TaskProfile` before it runs:

Feature	Small model if	Large model if
`context_tokens`	< 2,000	> 6,000
`n_segments` (role × source × stage cells)	< 10	> 30
`needs_causal_reasoning` (e.g. JD vs interview)	False	True
`last_eval_score` for this agent	⩾ 0.8	< 0.6
`stakes` (offer/comp decisions)	low	high

`hiring_intel/agents/router.py`

```
from dataclasses import dataclass
from pydantic import BaseModel
from typing import Literal
from hiring_intel.llm import CFG, call_structured
from hiring_intel.observability import DB

@dataclass
class TaskProfile:
    agent: str
    context_tokens: int
    n_segments: int
    needs_causal_reasoning: bool
```

```
    stakes: Literal["low", "high"]
    last_eval_score: float | None = None

class RouteDecision(BaseModel):
    tier: Literal["small", "large"]
    reason: str

def score(p: TaskProfile) -> float:
    s = 0.0
    s += 0.3 if p.context_tokens > 6000 else 0.15 if p.context_tokens > 2000
else 0
    s += 0.2 if p.n_segments > 30 else 0.1 if p.n_segments > 10 else 0
    s += 0.25 if p.needs_causal_reasoning else 0
    s += 0.15 if p.stakes == "high" else 0
    if p.last_eval_score is not None and p.last_eval_score < 0.6:
        s += 0.3                                    # escalate agents that
judged poorly last run
    return s

def route(p: TaskProfile, run_id: str, force_tier: str | None = None) -> str:
    if force_tier:                                  # used by benchmarks (all-
large baseline)
        tier, reason = force_tier, "forced"
    else:
        s = score(p)
        if s < 0.35:   tier, reason = "small", f"rule score {s:.2f}"
        elif s > 0.55: tier, reason = "large", f"rule score {s:.2f}"
        else:                                        # borderline: ask the
small model
            d, _ = call_structured(agent="router", run_id=run_id,
model=CFG["tiers"]["small"]["model"],
                system="You choose a model tier for an analytics sub-task.
Pick 'large' only if the task "
                       "needs multi-step causal reasoning over many
segments.",
                user=str(p.__dict__), schema=RouteDecision)
            tier, reason = d.tier, f"llm: {d.reason} (score {s:.2f})"
    model = CFG["tiers"][tier]["model"]
    DB.execute("INSERT INTO routing VALUES (?,?,?,?,?)", (run_id, p.agent,
tier, model, reason)); DB.commit()
    return model
```

Expected routing on the mock data: Panel Load and Pipeline Health go to the small model (mostly arithmetic interpretation); Rejection Pattern and Offer Insights go large (causal attribution, comp stakes); Sourcing depends on segment count.

Escalation loop. If the Evaluation Agent scores an agent's output below 0.6, the graph re-runs that agent once with `force_tier="large"`. Log both attempts; the before/after appears in the dashboard's routing table.

Step 6 — The five insight agents

Every insight agent runs the same four steps: compute features in pandas, retrieve context, call the routed model with a versioned prompt, then post-process (calibrate confidence, attach cost, validate). Write that once in a base class; each agent then supplies only its feature function, its query and its prompt.

6.1 Deterministic features — `hiring_intel/metrics.py`

```
import pandas as pd

def sourcing_features(d) -> pd.DataFrame:
    c = d.candidates.merge(d.requisitions[["req_id", "role"]], on="req_id")
    g = c.groupby(["role", "source"]).agg(applicants=("candidate_id",
"count"),
            hires=("outcome", lambda s: (s == "hired").sum()),
            reached_onsite=("current_stage", lambda s: s.isin(["onsite",
"offer"]).sum()))
    g["hire_rate"] = g.hires / g.applicants
    g["onsite_rate"] = g.reached_onsite / g.applicants
    return g.reset_index().round(3)

def rejection_features(d) -> pd.DataFrame:
    e = (d.stage_events.merge(d.candidates[["candidate_id", "req_id",
"source"]], on="candidate_id")
                       .merge(d.requisitions[["req_id", "role"]],
on="req_id"))
    rej = e[e.result == "reject"]
    by = rej.groupby(["role", "stage",
"rejection_reason"]).size().rename("rejections").reset_index()
    tot = e.groupby(["role", "stage"]).size().rename("entered").reset_index()
    out = by.merge(tot, on=["role", "stage"])
    out["share_of_stage"] = (out.rejections / out.entered).round(3)
```

```
    return out.sort_values("share_of_stage", ascending=False).head(25)   #
cap context size

def panel_load_features(d, weeks: int = 4) -> pd.DataFrame:
    i = d.interviews[d.interviews.scheduled_at >=
d.interviews.scheduled_at.max() - pd.Timedelta(weeks=weeks)]
    load =
i.groupby("interviewer_id").size().rename("interviews").reset_index()
    load = d.interviewers.merge(load, how="left").fillna({"interviews": 0})
    load["per_week"] = (load.interviews / weeks).round(2)
    load["share"] = (load.interviews / load.interviews.sum()).round(3)
    return load.sort_values("interviews", ascending=False)

def offer_features(d) -> pd.DataFrame:
    o = d.offers.merge(d.requisitions[["req_id", "role", "level"]],
on="req_id")
    o["ctc_ratio"] = (o.offered_ctc / o.expected_ctc).round(2)
    o["ratio_band"] = pd.cut(o.ctc_ratio, [0, 0.9, 1.0, 2], labels=["<0.9",
"0.9-1.0", ">1.0"])
    o["days_to_respond"] = (o.responded_at - o.extended_at).dt.days
    return (o.groupby(["role", "ratio_band", "status", "decline_reason"],
dropna=False, observed=True)
             .size().rename("n").reset_index())

def pipeline_features(d, sla: dict) -> dict:
    e = d.stage_events.copy()
    e["dwell"] = (e.exited_at - e.entered_at).dt.days
    e = e.merge(d.candidates[["candidate_id",
"req_id"]]).merge(d.requisitions[["req_id", "role"]])
    e["sla"] = e.stage.map(sla["stage_sla_days"])
    breaches = (e.assign(breach=e.dwell > e.sla).groupby(["role", "stage"])
                 .agg(median_dwell=("dwell", "median"), breach_rate=
("breach", "mean")).round(2).reset_index())
    today = d.stage_events.exited_at.max()
    r = d.requisitions[d.requisitions.status == "open"].copy()
    r["days_open"] = (today - r.opened_at).dt.days
    funnel = d.stage_events.groupby("stage").candidate_id.nunique().to_dict()
    return {"funnel": funnel, "breaches": breaches[breaches.breach_rate >
0.3],
            "aging_reqs": r[r.days_open > sla["max_days_open"]["default"]]
[["req_id", "role", "days_open"]]}
```

Unit-test these against `docs/planted_patterns.md`. If the referral hire rate for Backend is not about 3x job boards, fix the generator before you touch a prompt.

6.2 The base agent — `hiring_intel/agents/base.py`

```
import math, pathlib, pandas as pd
from hiring_intel.contracts import AgentOutput, LLMInsights
from hiring_intel.llm import call_structured
from hiring_intel.rag.retriever import retrieve
from hiring_intel.agents.router import TaskProfile, route

SYSTEM_COMMON = """You are a hiring analytics expert advising an Engineering
Manager.
Rules:
- Use ONLY numbers present in FEATURES or CONTEXT. Never invent figures.
- Each evidence item names the metric, its value and the segment it comes
from.
- Evidence from CONTEXT has source="rag" and ref=<chunk id in brackets>.
- A recommendation names WHO does WHAT by WHEN, or a measurable target.
- confidence_score: 0.9+ only if n>=50 in the segment and the effect is
large; <0.5 if n<15.
- alternative: a cheaper or faster way to reach a similar decision, with its
trade-off.
- Return 1-3 insights, most impactful first."""

class BaseInsightAgent:
    name: str; rag_query: str; causal: bool = False; stakes: str = "low";
schema = LLMInsights

    def features(self, data) -> pd.DataFrame | dict: raise
NotImplementedError
    def role_filter(self, data) -> str | None: return None

    def prompt(self) -> str:
        return (pathlib.Path(__file__).parent.parent / "prompts" / f"
{self.name}.md").read_text()

    def run(self, data, run_id: str, force_tier: str | None = None,
last_eval: float | None = None) -> AgentOutput:
        try:
            feats = self.features(data)
            feats_txt = feats.to_csv(index=False) if isinstance(feats,
```

```
pd.DataFrame) else \
                        "\n\n".join(f"## {k}\n{v.to_csv(index=False) if
hasattr(v, 'to_csv') else v}"
                                    for k, v in feats.items())
            chunks = retrieve(self.rag_query, self.name,
self.role_filter(data))
            ctx = "\n".join(f"[{cid}] {txt}" for cid, txt in chunks) or "(no
relevant context found)"
            profile = TaskProfile(self.name, context_tokens=len(feats_txt +
ctx) // 4,
                                  n_segments=feats_txt.count("\n"),
needs_causal_reasoning=self.causal,
                                  stakes=self.stakes,
last_eval_score=last_eval)
            model = route(profile, run_id, force_tier)
            parsed, cost = call_structured(agent=self.name, run_id=run_id,
model=model,
                system=SYSTEM_COMMON + "\n\n" + self.prompt(),
                user=f"FEATURES (computed from ATS
data):\n{feats_txt}\n\nCONTEXT (retrieved):\n{ctx}",
                schema=self.schema)
            insights = [self._calibrate(i, feats) for i in parsed.insights]
            return AgentOutput(agent=self.name, run_id=run_id,
insights=insights, cost_of_insight=cost,
                               retrieved_chunks=[c for c, _ in chunks],
                               status="ok" if chunks else "degraded")
        except Exception as e:                      # never let one agent
kill the run
            return AgentOutput(agent=self.name, run_id=run_id,
status="failed", error=f"{type(e).__name__}: {e}")

    def _calibrate(self, ins, feats):
        """Cap the LLM's self-reported confidence by evidence strength."""
        n_ev = len(ins.evidence); has_rag = any(e.source == "rag" for e in
ins.evidence)
        cap = min(0.95, 0.5 + 0.1 * n_ev + (0.1 if has_rag else 0))
        ins.confidence_score = round(min(ins.confidence_score, cap), 2)
        return ins
```

The confidence cap is a simple, defensible calibration: the model cannot claim 0.9 on one data point with no benchmark. In Week 2, check calibration by plotting confidence against the judge's grounding score on the golden set.

6.3 One agent in full — Rejection Pattern

```
# hiring_intel/agents/rejection.py
from hiring_intel.agents.base import BaseInsightAgent
from hiring_intel.metrics import rejection_features

class RejectionPatternAgent(BaseInsightAgent):
    name = "rejection"
    rag_query = "rejection history by stage and reason; JD quality vs
interview calibration"
    causal = True                    # JD problem vs interviewing problem
needs reasoning
    def features(self, data): return rejection_features(data)
```

`prompts/rejection.md`:

```
TASK: Find where and why candidates are rejected, by role and stage.
Decide for each top pattern whether it is a JD/sourcing problem or an
interviewing problem:
- High rejection at `screen` with reason "JD mismatch" or "experience gap" ->
JD or sourcing problem.
- High rejection at technical stages concentrated in one source -> sourcing
quality problem.
- High rejection at technical stages across all sources -> bar or interviewer
calibration problem.
Compare with prior-quarter rejection history in CONTEXT when available.
Start each title with the root cause in brackets: [jd], [sourcing],
[interview_calibration] or [unclear].
```

6.4 The other four agents

Agent	Features	RAG query	Prompt focus	Typical tier
Sourcing Quality	`sourcing_features`	"source conversion benchmark {role}"	Rank channels per role by hire rate vs benchmark; name channels to cut or double down on, with budget shift	small/large by segments
Panel Load Balancer	`panel_load_features`	"interviewer load policy"	Flag anyone over `max_interviews_per_week`; propose a named rebalance (move N slots from A to B, same team/level)	small

Agent	Features	RAG query	Prompt focus	Typical tier
Offer Insights	`offer_features`	"offer decline compensation band {role}"	Attribute declines to comp, timing or competing offers; quantify the CTC-ratio threshold; suggest band or speed changes	large (stakes high)
Pipeline Health	`pipeline_features` + SLA config	"stage SLA targets time to fill"	Name SLA-breaching stage × role pairs and aging reqs; propose one fix per breach with an owner	small

For the Panel Load Balancer, extend the contract with a structured plan so the dashboard can render it as a table:

```
class Move(BaseModel):
    from_interviewer: str; to_interviewer: str; slots_per_week: int

class PanelInsight(Insight):
    rebalance_plan: list[Move] = []
```

Then set that agent's `schema` to a `LLMInsights` variant with `list[PanelInsight]`. Keep the five base fields untouched so the evaluator and the UI need no special case.

Step 7 — LangGraph orchestrator with error handling

The graph is: ingest → five insight agents in parallel → evaluate → re-run low scorers once on the large model → optimize → persist. A failed agent writes a `failed` output into state instead of raising, so agent three crashing never drops the run.

`hiring_intel/graph.py`

```
import operator, uuid, yaml
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END
from hiring_intel.ingest import load_all
from hiring_intel.agents.sourcing import SourcingQualityAgent
from hiring_intel.agents.rejection import RejectionPatternAgent
from hiring_intel.agents.panel_load import PanelLoadAgent
from hiring_intel.agents.offers import OfferInsightsAgent
from hiring_intel.agents.pipeline_health import PipelineHealthAgent
from hiring_intel.agents.evaluator import evaluate_outputs
```

```
from hiring_intel.agents.optimizer import optimize
from hiring_intel.observability import DB

AGENTS = {a.name: a for a in [SourcingQualityAgent(),
RejectionPatternAgent(), PanelLoadAgent(),
                              OfferInsightsAgent(), PipelineHealthAgent()]}

def merge_outputs(old: dict, new: dict) -> dict:     # reducer: later write
for an agent wins
    return {**old, **new}

class RunState(TypedDict, total=False):
    run_id: str
    data_path: str
    data: object
    outputs: Annotated[dict, merge_outputs]          # agent name ->
AgentOutput
    scores: dict                                     # agent name ->
EvalResult
    retried: list[str]
    suggestions: list
    errors: Annotated[list, operator.add]

def ingest_node(s: RunState):
    return {"data": load_all(s["data_path"]), "run_id": s.get("run_id") or
uuid.uuid4().hex[:8]}

def make_agent_node(name):
    def node(s: RunState):
        out = AGENTS[name].run(s["data"], s["run_id"])
        errs = [f"{name}: {out.error}"] if out.status == "failed" else []
        return {"outputs": {name: out}, "errors": errs}
    return node

def evaluate_node(s: RunState):
    return {"scores": evaluate_outputs(s["outputs"], s["data"], s["run_id"])}

def needs_retry(s: RunState) -> str:
    low = [a for a, r in s["scores"].items() if r.overall < 0.6 and a not in
s.get("retried", [])]
    return "retry" if low else "optimize"

def retry_node(s: RunState):
```

```
    low = [a for a, r in s["scores"].items() if r.overall < 0.6]
    new = {a: AGENTS[a].run(s["data"], s["run_id"], force_tier="large") for a
in low}
    return {"outputs": new, "retried": low}

def optimize_node(s: RunState):
    return {"suggestions": optimize(s["run_id"])}

def persist_node(s: RunState):
    for a, out in s["outputs"].items():
        DB.execute("INSERT INTO agent_outputs VALUES (?,?,?,?)",
(s["run_id"], a, out.status, out.model_dump_json()))
    DB.commit(); return {}

g = StateGraph(RunState)
g.add_node("ingest", ingest_node)
for name in AGENTS:
    g.add_node(name, make_agent_node(name))
    g.add_edge("ingest", name)                      # fan-out: all five run
in parallel
    g.add_edge(name, "evaluate")                    # fan-in: evaluate waits
for all five
g.add_node("evaluate", evaluate_node)
g.add_node("retry", retry_node)
g.add_node("optimize", optimize_node)
g.add_node("persist", persist_node)
g.add_edge(START, "ingest")
g.add_conditional_edges("evaluate", needs_retry, {"retry": "retry",
"optimize": "optimize"})
g.add_edge("retry", "evaluate")                     # re-score once;
`retried` stops a loop
g.add_edge("optimize", "persist")
g.add_edge("persist", END)
APP = g.compile()

def run(data_path="data/raw", run_id=None):
    return APP.invoke({"data_path": data_path, "run_id": run_id, "outputs":
{}, "errors": [], "retried": []})
```

`scripts/run_pipeline.py` just calls `run()` and prints a summary. Export the graph picture for your docs with `APP.get_graph().draw_mermaid()`.

Error handling at each hop

Hop	Failure	Handling
Ingest	Bad rows, missing columns	Quarantine rows; hard-fail only on a missing table
Retrieval	Empty result or Chroma error	Continue with "(no context)"; status `degraded`; confidence capped
LLM call	Timeout, 429, 5xx	LiteLLM `num_retries=2` with backoff; then `failed` output
JSON	Schema violation	One repair turn with the validation error; then `failed`
Agent	Any exception	Caught in `BaseInsightAgent.run`; recorded in `errors`
Evaluate	Judge fails	Schema and grounding checks still run; judge score `null`
Dashboard	Failed agent	Shown as a red card with the error, never hidden

Test the failure path on purpose: set an invalid model name for one agent and confirm the other four still reach the dashboard. Show that in the demo video; it directly answers the "errors handled" criterion.

Step 8 — Evaluation Agent, golden set and LLM-as-judge

Evaluate in three layers, cheapest first: Pydantic schema checks (free), a deterministic grounding check that every cited number exists in the features (free), then an LLM judge scoring actionability, grounding and hallucination risk. The overall score blends them; anything under 0.6 is retried on the large model, and anything flagged as hallucinated is hidden behind a warning in the UI.

8.1 The golden set — `data/golden/golden_set.jsonl`

Write 15–20 scenarios, three or four per insight agent. Each holds a small, hand-made feature table, the finding a good analyst would reach, and what must not appear.

```
{"id": "rej-03", "agent": "rejection",
 "features_csv":
"role,stage,rejection_reason,rejections,entered,share_of_stage\nMobile,screen
,JD mismatch,96,120,0.8\nMobile,tech_1,coding,6,24,0.25",
 "context": [["rej_hist_q1#0", "Q1: Mobile screen rejection 35%, mostly
experience gap."]],
 "expected": {"segment": {"role": "Mobile", "stage": "screen"}, "root_cause":
"jd",
              "must_mention": ["JD"], "min_confidence": 0.6},
 "must_not": ["interviewer calibration"]}
```

Cover the easy cases (one clear pattern), the hard ones (two competing explanations), and traps: a segment with n = 4 (the right answer is low confidence), a table with no real pattern (the right answer is "no action"), and context that contradicts the data (data should win).

8.2 `hiring_intel/agents/evaluator.py`

```
import re
from pydantic import BaseModel, Field
from hiring_intel.llm import CFG, call_structured

class JudgeScore(BaseModel):
    actionability: int = Field(ge=1, le=5)     # owner + action + measurable
target?
    grounding: int = Field(ge=1, le=5)         # every claim traceable to
FEATURES/CONTEXT?
    hallucination_risk: int = Field(ge=1, le=5)  # 5 = high risk
    rationale: str

class EvalResult(BaseModel):
    agent: str
    schema_ok: bool
    numbers_grounded: float          # share of evidence values found in
features
    refs_valid: bool                 # every rag ref was actually retrieved
    judge: JudgeScore | None = None
    golden_match: float | None = None
    overall: float
    flags: list[str] = []

RUBRIC = """You are a strict reviewer of hiring analytics. Score each insight
```

```
against the SOURCE DATA.
actionability 5 = names who, what and a measurable target; 1 = generic
advice.
grounding 5 = every number and claim appears in SOURCE DATA; 1 = mostly
unsupported.
hallucination_risk 5 = contains figures, benchmarks or causes not in SOURCE
DATA; 1 = none.
Be harsh: a plausible but unsupported benchmark counts as hallucination."""

def _numbers(text: str) -> set[float]:
    return {round(float(x), 2) for x in re.findall(r"-?\d+\.?\d*", text)}

def grounding_check(out, features_txt: str) -> tuple[float, bool]:
    src = _numbers(features_txt); vals, hits = 0, 0
    for ins in out.insights:
        for ev in ins.evidence:
            if ev.source == "data" and isinstance(ev.value, (int, float)):
                vals += 1
                v = round(float(ev.value), 2)
                hits += any(abs(v - s) <= 0.01 or abs(v * 100 - s) <= 0.5 for
s in src)  # allow 0.8 vs 80%
    refs_ok = all(ev.ref in out.retrieved_chunks for i in out.insights for ev
in i.evidence if ev.source == "rag")
    return (hits / vals if vals else 0.0), refs_ok

def judge(out, features_txt, run_id) -> JudgeScore:
    s, _ = call_structured(agent="evaluator", run_id=run_id,
model=CFG["judge"]["model"], system=RUBRIC,
        user=f"SOURCE
DATA:\n{features_txt}\n\nINSIGHTS:\n{out.model_dump_json(include=
{'insights'})}",
        schema=JudgeScore, temperature=0)
    return s

def combine(schema_ok, grounded, refs_ok, j: JudgeScore | None, golden=None)
-> tuple[float, list[str]]:
    if not schema_ok: return 0.0, ["schema"]
    flags = []
    if grounded < 0.8: flags.append("ungrounded_numbers")
    if not refs_ok: flags.append("invalid_rag_ref")
    jscore = 0.5 if j is None else ((j.actionability + j.grounding + (6 -
j.hallucination_risk)) / 15)
    if j and j.hallucination_risk >= 4: flags.append("hallucination_risk")
```

```
    parts = [0.35 * grounded, 0.15 * refs_ok, 0.5 * jscore]
    score = sum(parts) if golden is None else 0.7 * sum(parts) + 0.3 * golden
    return round(score, 2), flags

def evaluate_outputs(outputs: dict, data, run_id) -> dict[str, EvalResult]:
    from hiring_intel.graph import AGENTS
    results = {}
    for name, out in outputs.items():
        if out.status == "failed":
            results[name] = EvalResult(agent=name, schema_ok=False,
numbers_grounded=0, refs_valid=False,
                                       overall=0.0, flags=["agent_failed"]);
continue
        feats = AGENTS[name].features(data)
        ftxt = feats.to_csv(index=False) if hasattr(feats, "to_csv") else
str(feats)
        grounded, refs_ok = grounding_check(out, ftxt)
        try: j = judge(out, ftxt, run_id)
        except Exception: j = None
        overall, flags = combine(True, grounded, refs_ok, j)
        results[name] = EvalResult(agent=name, schema_ok=True,
numbers_grounded=round(grounded, 2),
                                   refs_valid=refs_ok, judge=j,
overall=overall, flags=flags)
    return results
```

8.3 Golden-set runner — `scripts/run_eval.py`

For each scenario, run the agent's prompt on the scenario's features and context (skip ingest and retrieval), then score:

- **segment hit** — the top insight's evidence contains the expected segment (1/0);
- **must_mention / must_not** — keyword checks on recommendation and title;
- **confidence check** — at or above `min_confidence`, or below it for the low-n traps;
- **judge score** — the rubric above.

`golden_match` = mean of the first three; write a CSV per run to `data/eval/` with the prompt version and model, so you can compare prompt v1 vs v2 and small vs large. That CSV is also your A/B harness for prompts, which covers one of the brief's stretch goals.

Validate the judge too. Hand-score 10 outputs yourself, then report agreement with the judge (for example, "within 1 point on 8/10"). One sentence of this in the eval notes earns a lot of trust in the 25% quality area. Use a different or larger model for the judge than for the generator, so it does not grade its own style.

Step 9 — Optimization Agent and the before/after levers

The Optimization Agent reads the `llm_calls` table after each run, finds the expensive or slow agents, and proposes changes in the same contract shape. Prove it works with a benchmark script that measures at least two levers against a baseline, and always report quality next to cost, so a saving that hurts quality is visible.

9.1 `hiring_intel/agents/optimizer.py`

```
import pandas as pd
from hiring_intel.observability import DB
from hiring_intel.llm import CFG, call_structured
from hiring_intel.contracts import LLMInsights

def run_stats(run_id: str) -> pd.DataFrame:
    df = pd.read_sql("SELECT * FROM llm_calls WHERE run_id = ?", DB, params=
(run_id,))
    return (df.groupby(["agent", "model"])
              .agg(calls=("usd", "size"), usd=("usd", "sum"), in_tok=
("input_tokens", "sum"),
                   out_tok=("output_tokens", "sum"), p50_ms=("latency_ms",
"median"), cached=("cached", "sum"))
              .reset_index().round(5))

def rule_suggestions(stats: pd.DataFrame, scores: dict | None = None) ->
list[str]:
    tips, small = [], CFG["tiers"]["small"]["model"]
    for r in stats.itertuples():
        if r.in_tok > 6000: tips.append(f"{r.agent}: {r.in_tok} input tokens;
cap feature rows or prune columns")
        if r.model != small and scores and scores.get(r.agent) and
scores[r.agent].overall > 0.85:
            tips.append(f"{r.agent}: scores {scores[r.agent].overall} on
{r.model}; trial the small model")
        if r.calls > 1 and r.cached == 0: tips.append(f"{r.agent}: {r.calls}
calls, no cache hits; check repair loops")
```

```
    return tips

def optimize(run_id: str, scores: dict | None = None):
    stats = run_stats(run_id)
    tips = rule_suggestions(stats, scores)
    out, _ = call_structured(agent="optimizer", run_id=run_id,
model=CFG["tiers"]["small"]["model"],
        system="You are a cost engineer for an LLM pipeline. Turn the stats
and rule hits into 1-3 "
               "concrete optimizations. Evidence = the stats rows.
Alternative = the option you rejected.",
        user=f"STATS:\n{stats.to_csv(index=False)}\n\nRULE HITS:\n" +
"\n".join(tips), schema=LLMInsights)
    return out.insights
```

Pass `s["scores"]` from the graph's `optimize_node` so the agent never suggests downgrading a model that is scoring poorly.

9.2 The levers to measure

Lever	Baseline	Optimized	How to switch
A. Model routing	Every agent on the large tier	Router picks per agent	`force_tier="large"` vs router
B. Context compression	Full feature tables, all columns, 4 RAG chunks	Top 25 rows, pruned columns, 2-decimal rounding, top_k 2	`variant` flag in each agent's `features()`
C. Caching	Cache off	LLM response cache + `lru_cache` retrieval	`use_cache=False` vs `True` on a repeated run
D. (optional) Rules-first	LLM writes every insight	Panel Load uses pure pandas thresholds; LLM only phrases the plan	Agent flag

The brief needs two; A and B are the strongest because they keep their savings on fresh data, while C mostly helps repeated runs and the demo.

9.3 `scripts/benchmark_levers.py`

```
import pandas as pd, statistics as st
from hiring_intel.graph import run
from hiring_intel.observability import DB
from scripts.run_eval import golden_score   # mean golden_match for a variant

VARIANTS = {"baseline_all_large": dict(force_tier="large", compress=False,
cache=False),
            "A_routing":          dict(force_tier=None,    compress=False,
cache=False),
            "A+B_compressed":     dict(force_tier=None,    compress=True,
cache=False),
            "A+B+C_cached":       dict(force_tier=None,    compress=True,
cache=True)}

rows = []
for name, cfg in VARIANTS.items():
    usd, lat = [], []
    for rep in range(3):                                    # 3 reps to
smooth latency noise
        rid = f"{name}-{rep}"; run(run_id=rid, **cfg)
        df = pd.read_sql("SELECT usd, latency_ms FROM llm_calls WHERE
run_id=?", DB, params=(rid,))
        usd.append(df.usd.sum()); lat.append(df.latency_ms.sum())
    rows.append({"variant": name, "usd_per_run": round(st.mean(usd), 4),
                 "latency_s": round(st.median(lat) / 1000, 1),
"golden_score": golden_score(**cfg)})
report = pd.DataFrame(rows)
base = report.iloc[0]
report["usd_saving_%"] = ((1 - report.usd_per_run / base.usd_per_run) *
100).round(1)
report.to_markdown("docs/optimization_report.md", index=False)
print(report)
```

Thread the `compress` and `cache` flags from `run()` into each agent and into `call_structured`. Latency sums agent time; because the five agents run in parallel, also record wall-clock time per run for the dashboard.

The report you submit is that generated table plus two sentences per lever: what changed, and whether the golden score held. Do not quote expected savings before you measure them; the brief rewards seeing and moving the cost, not a target percentage.

Step 10 — Streamlit dashboard

Design for the "understand it in under a minute" test: a KPI row, the funnel, then one card per insight sorted by severity × confidence, with cost and evaluation tucked into tabs. An EM should see the top three actions without scrolling.

Tab	Shows
Overview	KPI tiles (open reqs, median time-to-fill, offer accept rate, SLA breaches), funnel chart, top 3 insights
Insights	One card per insight: recommendation, evidence table, confidence bar, eval flags, alternative, cost
Evaluation	Per-agent judge scores, grounding %, golden-set results by prompt version
Cost & latency	USD and tokens per agent, p50 latency, routing decisions, optimization report
Run log	Raw `llm_calls` and errors for debugging

`app/dashboard.py`

```
import json, pandas as pd, plotly.express as px, sqlite3, streamlit as st
from hiring_intel.contracts import AgentOutput
from hiring_intel.ingest import load_all

st.set_page_config(page_title="Hiring Intelligence", layout="wide")
DB = sqlite3.connect("data/runs.db", check_same_thread=False)

runs = pd.read_sql("SELECT DISTINCT run_id FROM agent_outputs",
DB).run_id.tolist()
run_id = st.sidebar.selectbox("Run", runs[::-1])
if st.sidebar.button("Run pipeline now"):
    from hiring_intel.graph import run
    with st.spinner("Running 8 agents..."): run(); st.rerun()

outs = [AgentOutput.model_validate_json(j) for j in
        pd.read_sql("SELECT json FROM agent_outputs WHERE run_id=?", DB,
params=(run_id,)).json]
calls = pd.read_sql("SELECT * FROM llm_calls WHERE run_id=?", DB, params=
```

```
(run_id,))
routes = pd.read_sql("SELECT * FROM routing WHERE run_id=?", DB, params=
(run_id,))
data = load_all("data/raw")

t1, t2, t3, t4, t5 = st.tabs(["Overview", "Insights", "Evaluation", "Cost &
latency", "Run log"])

with t1:
    c = st.columns(4)
    c[0].metric("Open reqs", int((data.requisitions.status == "open").sum()))
    c[1].metric("Offer accept rate", f"{(data.offers.status ==
'accepted').mean():.0%}")
    c[2].metric("Run cost", f"${calls.usd.sum():.4f}")
    c[3].metric("Agents OK", f"{sum(o.status == 'ok' for o in
outs)}/{len(outs)}")
    funnel =
data.stage_events.groupby("stage").candidate_id.nunique().reindex(
        ["applied", "screen", "tech_1", "tech_2", "onsite",
"offer"]).reset_index()
    st.plotly_chart(px.funnel(funnel, x="candidate_id", y="stage"),
use_container_width=True)
    top = sorted([(o.agent, i) for o in outs for i in o.insights],
                 key=lambda t: ({"high": 3, "medium": 2, "low": 1}
[t[1].severity], t[1].confidence_score), reverse=True)[:3]
    for agent, i in top:
        st.info(f"**{i.title}** ({agent}, confidence
{i.confidence_score:.0%})\n\n{i.recommendation}")

with t2:
    for o in outs:
        st.subheader(o.agent.replace("_", " ").title())
        if o.status == "failed": st.error(o.error); continue
        if o.status == "degraded": st.warning("No RAG context retrieved;
confidence capped")
        for i in o.insights:
            with st.expander(f"{i.title} — {i.confidence_score:.0%}",
expanded=True):
                st.write(i.recommendation)
                st.progress(i.confidence_score)
                st.dataframe(pd.DataFrame([e.model_dump() for e in
i.evidence]), hide_index=True)
                st.caption(f"Alternative: {i.alternative.option} —
```

```
{i.alternative.trade_off}")
        if o.cost_of_insight:
            k = o.cost_of_insight
            st.caption(f"{k.model} · {k.input_tokens}+{k.output_tokens}
tokens · ${k.usd:.4f} · {k.latency_ms} ms")

with t4:
    by = calls.groupby("agent").agg(usd=("usd", "sum"), p50_ms=("latency_ms",
"median")).reset_index()
    a, b = st.columns(2)
    a.plotly_chart(px.bar(by, x="agent", y="usd", title="USD per agent"),
use_container_width=True)
    b.plotly_chart(px.bar(by, x="agent", y="p50_ms", title="Median latency
(ms)"), use_container_width=True)
    st.dataframe(routes, hide_index=True)
    try: st.markdown(open("docs/optimization_report.md").read())
    except FileNotFoundError: st.caption("Run scripts/benchmark_levers.py to
generate the report")

with t5:
    st.dataframe(calls, hide_index=True)
```

Fill the Evaluation tab (t3) the same way: persist `EvalResult` objects to an `eval_results` table in `persist_node`, then show a per-agent bar of `overall` and a table of flags. Run it with `streamlit run app/dashboard.py`.

Human feedback hook. Add thumbs-up/down buttons per insight that write to a `feedback` table. Thumbs-up insights get indexed into the RAG store as `past_insight`, which closes the loop from Step 4.

Step 11 — Tests, docs package and demo

Tests that matter (`tests/`)

- `test_metrics.py` — each feature function recovers its planted pattern (referral ≈ 3x job board for Backend, and so on).
- `test_contracts.py` — generic recommendations, confidence > 1 and empty evidence are rejected.
- `test_grounding.py` — an insight citing a number absent from the features scores below 0.8.

- `test_graph_resilience.py` — monkeypatch one agent to raise; the run finishes with four `ok` and one `failed`.
- `test_router.py` — small profiles route small, causal high-stakes ones route large.

Mock `litellm.completion` in unit tests so CI costs nothing; run the golden set as a separate, paid script.

Documentation package (`docs/`)

File	Contents
`architecture.md`	The diagram from the Overview, the LangGraph Mermaid export, data flow and error handling table
`contracts.md`	`Insight` and `AgentOutput` fields, one real JSON example per agent
`eval_notes.md`	Golden-set design, rubric, judge-vs-human agreement, top_k/threshold tuning, known failure cases
`optimization_report.md`	Generated before/after table + two sentences per lever
`reflection.md`	One page: why LangGraph, why rules-first routing, why pandas-first features, what you would change
`README.md`	Setup in 5 commands, `.env` keys, how to regenerate data, run, evaluate, benchmark, launch UI

Demo video (5–10 minutes)

1. Problem and architecture diagram (1 min).
2. Live run from the dashboard button; show parallel agents in the logs (1.5 min).
3. Trace one decision end to end: Mobile screen rejection → features row → retrieved Q1 history chunk → routed to large model → insight with root cause `[jd]` → judge scores → card in UI (3 min).
4. Evaluation tab and golden-set results, including one failure you found and fixed (1.5 min).
5. Cost tab: routing table and the before/after report (1.5 min).

6. Kill one agent and show the run survives (30 s).

Step 12 — Two-week plan and checklist mapping

Days	Build	Done when
1	Repo, config, mock generator, planted patterns	Six CSVs; `test_metrics` passes
2	Contracts, `llm.py`, observability	One structured call logged with USD
3	Knowledge base + Chroma index + retriever	10-query hit@k table
4	Base agent + all five insight agents	Each returns a valid `AgentOutput`
4–5	Router + LangGraph graph	First full run; **Interim Connect 1**: diagram, contracts, top 3 blockers
6–7	Golden set (15–20) + evaluator + judge validation	Golden CSV per prompt version
8	Retry loop + optimizer + benchmark script	Before/after for levers A and B
9	Dashboard	All five tabs render on a fresh run; **Interim Connect 2**
10–11	Tests, docs, README dry run on a clean venv	Setup works from scratch
12	Record demo, submit	Checklist below all ticked

Submission checklist item	Where it is satisfied
Shared output contract on all agents	Step 3 `Insight` / `AgentOutput`
Orchestration end to end on mock data	Step 7 `graph.py` + Step 2 data
RAG grounding at least two agents	Step 4 + `refs_valid` check in Step 8
Evaluation vs golden set with LLM judge	Step 8
Routing live and logged	Step 5 `routing` table

Submission checklist item	Where it is satisfied
Two levers with before/after	Step 9 benchmark report
Dashboard: funnel, insights, cost, latency	Step 10
Docs package	Step 11
Demo video	Step 11 script

Stretch options, if time remains: expose `MockATSClient` and the retriever as MCP tools so any MCP client can call them; train a small scikit-learn or DistilBERT classifier on rejection-reason text and use it instead of the LLM for labelling (a third cost lever); and promote the golden-set CSVs into a prompt A/B report.